

Documentación

Meteo Map



Autores

Emiliano Recuenco López (868419)

José Miguel Quílez Vergara (873499)

Adrián Navarro Marín (874912)

Jorge Lucas Ferrer (839316)

Índice

1. URLs de acceso	2
2. Credenciales de acceso	2
3. Diagrama de la arquitectura.....	2
4. Fuentes de datos abiertos	3
4.1. Openmeteo	3
4.2. Aemet.....	3
5. Módulos utilizados para la implementación del back-end.....	4
6. Enlaces al Swagger de la API	5
7. Enlaces al prototipado de la solución	5
8. Tecnología utilizada para el front-end	5
9. Modelo de IA utilizado	6
10. Descripción de la validación y pruebas realizadas	7
11. Mejoras implementadas	7
12. Valoración global del proyecto.....	9
13. Mejoras propuestas	10

1. URLs de acceso

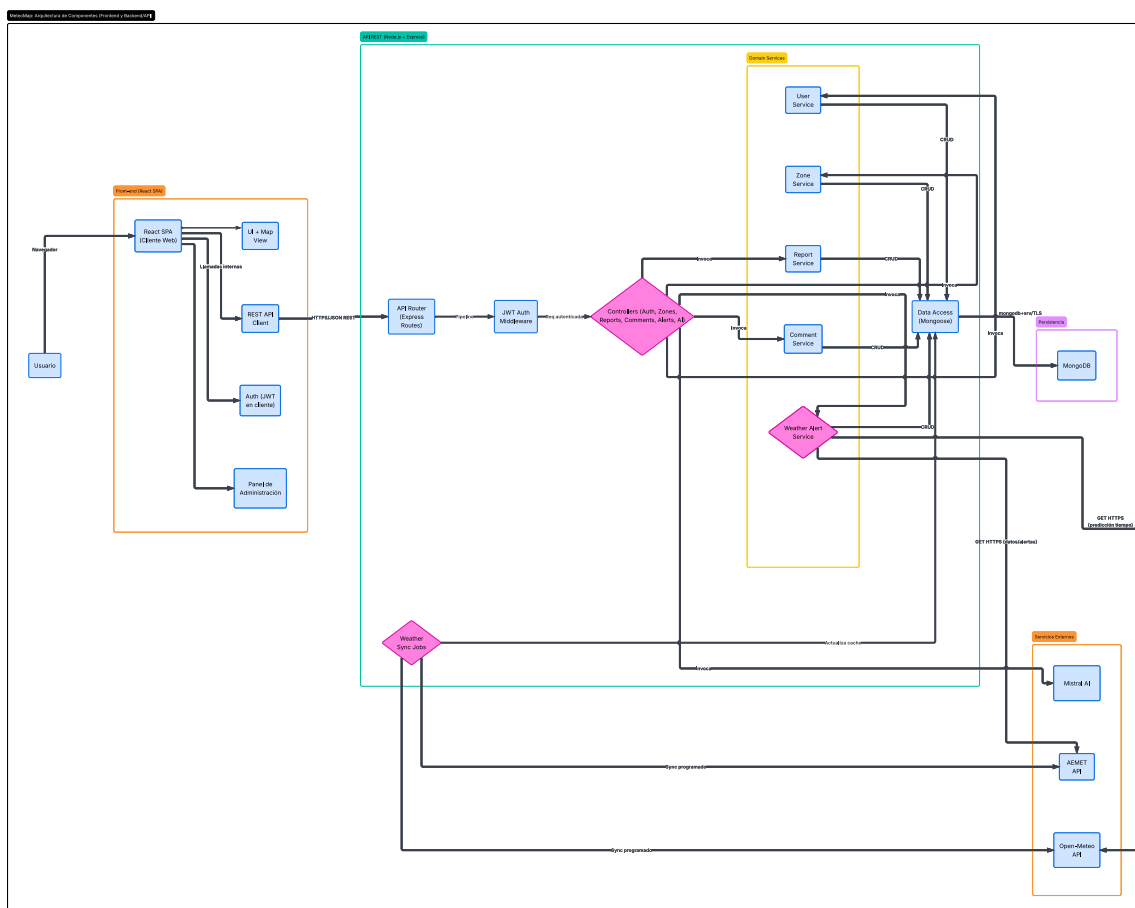
Tipo	URL
API (Swagger)	https://project-bpbih.vercel.app/docs
Front-end	https://meteo-map-frontend.vercel.app/

2. Credenciales de acceso

Rol	Usuario	Contraseña
ADMIN	admin@example.com	Password123#01
USER	testuser@example.com	Password123#01

3. Diagrama de la arquitectura

La arquitectura de la solución se organiza en dos capas principales: un front-end en React que consume una API REST desarrollada en Node.js con Express. El backend se apoya en MongoDB para persistencia, Mongoose para el modelado de datos, y una serie de servicios de dominio para zonas, reportes, comentarios, usuarios, alertas meteorológicas e IA.



4. Fuentes de datos abiertos

Se han utilizado varias fuentes abiertas. AEMET proporciona las alertas meteorológicas oficiales a través de su API de avisos, integrada en el backend mediante el servicio de alertas y almacenada en MongoDB con caché para evitar consultas repetidas. Open-Meteo se utiliza para obtener el estado meteorológico actual y la previsión por coordenadas geográficas; esos datos se sincronizan por zonas y se guardan en la caché meteorológica de cada zona. Además, la aplicación incorpora zonas geográficas de montaña y parques naturales almacenadas en la propia base de datos, y reportes y comentarios generados por la comunidad como fuente de información colaborativa.

4.1. Openmeteo

URL: <https://open-meteo.com/en/docs>

- Openmeteo se ha integrado para recopilar métricas y predicciones meteorológicas de las distintas zonas definidas en la aplicación. Concretamente, se emplea tanto para obtener predicciones a 12 horas desde el momento de la consulta como para recuperar información meteorológica en tiempo real. Para ello, se aprovecha una funcionalidad de la API que permite realizar consultas mediante listas de coordenadas geográficas en lugar de una única ubicación, optimizando así la sincronización de datos de múltiples zonas. Esta diferenciación entre datos actuales y predicciones permite representar la información de forma adecuada tanto en el dashboard personal como en la pantalla lateral de zonas.

4.2. Aemet

URL: <https://opendata.aemet.es/dist/index.html>

- Aemet se utiliza exclusivamente para la obtención de alertas y avisos meteorológicos oficiales emitidos por la Agencia Estatal de Meteorología. Estos avisos se incorporaron con el objetivo de destacar situaciones meteorológicas relevantes que puedan suponer un riesgo importante para los usuarios.

Sin embargo, ambas APIs presentan limitaciones relacionadas con el número de peticiones permitidas. En determinadas situaciones, las solicitudes pueden devolver errores asociados a un exceso de peticiones o incluso provocar bloqueos temporales de determinadas direcciones IP. Para reducir la dependencia de servicios externos y mejorar tanto la estabilidad como la latencia del sistema, se decidió implementar un sistema de caché. En el caso de las alertas meteorológicas, la información se almacena durante 30 minutos, mientras que las métricas climatológicas de las zonas se conservan en caché durante 2 horas. Además, para las alertas oficiales se desarrolló un mecanismo adicional que elimina automáticamente aquellas cuyo periodo de validez ya ha expirado.

Además, se implementó un servicio programado de actualización automática encargado de mantener sincronizada la información meteorológica. Este sistema ejecuta tareas periódicas en segundo plano que actualizan los datos climatológicos de las distintas zonas.

5. Módulos utilizados para la implementación del back-end

Módulo	Descripción
Node.js	Runtime de JavaScript para ejecutar el servidor backend
Express	Framework web minimalista para crear la API REST
MongoDB	Base de datos NoSQL para almacenar documentos
Mongoose	ODM (Object Data Modeling) para definir esquemas y operar con documentos de MongoDB
bcrypt	Librería para cifrado seguro de contraseñas
jsonwebtoken (JWT)	Creación y validación de tokens para autenticación
cors	Middleware para controlar el acceso desde diferentes orígenes
helmet	Middleware para endurecer la seguridad HTTP del servidor
cookie-parser	Middleware para parsear y gestionar cookies
express-validator	Validación y sanitización de datos en peticiones HTTP
morgan	Logging automático de peticiones HTTP
winston	Logger flexible para registrar eventos y errores
node-cron	Ejecutor de tareas programadas (cron jobs)
nodemailer	Envío de correos electrónicos (recuperación de contraseña)
zod	Validación y transformación de esquemas de datos
swagger-jsdoc	Generación de documentación OpenAPI desde comentarios JSDoc
swagger-ui-express	Interfaz gráfica interactiva para la documentación de la API
OpenAI SDK	Cliente compatible para conectar con el modelo de IA (Mistral)

google-auth-library	SDK oficial de Google para la validación y verificación segura de tokens en flujos OAuth2.
dotenv	Componente para la carga y gestión segura de variables de entorno (.env).
express-rate-limit & rate-limit-mongo	Sistema de seguridad para mitigar ataques DoS mediante el control y limitación del ratio de peticiones HTTP persistido en BD.
xml2js	Parser para transformar estructuras de datos XML procedentes de feeds externos en objetos JSON nativos.
nodemon	Herramienta de automatización en entorno de desarrollo para el reinicio en caliente (<i>hot-reload</i>) del servidor.

6. Enlaces al Swagger de la API

La documentación interactiva del API está disponible en <https://project-bpbih.vercel.app/docs>.

Esta documentación está generada desde la definición OpenAPI del backend y describe los endpoints de autenticación, usuarios, zonas, reportes, comentarios, chat con IA y administración. La configuración del Swagger se encuentra en swagger.js.

7. Enlaces al prototipado de la solución

- **Prototipo de interfaz (Figma):** <https://pony-glow-83451542.figma.site>

8. Tecnología utilizada para el front-end

Módulo	Descripción
React	Framework principal para construir la interfaz de usuario con componentes
React DOM	Librería para renderizar componentes React al DOM
Vite	Bundler y servidor de desarrollo ultrarrápido
Recharts	Sistema de renderizado de gráficos dinámicos para la analítica de datos meteorológicos.
Tailwind CSS	Framework CSS utility-first para estilos rápidos
TypeScript	Lenguaje tipado que compila a JavaScript
react-leaflet	Bindings React para Leaflet

shadcn/ui	Componentes reutilizables contruidos sobre Radix UI + Tailwind
@react-oauth/google	Integración OAuth con Google
leaflet	Librería de mapas interactivos (geolocalización, marcadores)
Cypress	Testing E2E (end-to-end)
recharts	Gráficos React basados en componentes (LineChart, BarChart, etc.)
radix-ui	Paquete general de Radix UI
sonner	Sistema de notificaciones tipo toast (éxito, error, etc.)
react-leaflet	Bindings React para Leaflet

9. Modelo de IA utilizado

El modelo utilizado para el asistente de IA es **mistral-small-latest**, integrado a través del SDK compatible de OpenAI. Se ha elegido por su buena relación entre capacidad de razonamiento, rendimiento en tareas de asistencia contextual y especialmente su bajo coste: los únicos límites que impone la API de Mistral sobre este modelo son de 625000 tokens por minuto (a diferencia de otros servicios como la API de Google Gemini, que proporciona tan sólo 5 peticiones / minuto con su modelo gemini-3-flash).

El asistente de IA se emplea para consultar datos proporcionados por la aplicación de forma más directa y accesible, así como para obtener ayuda o instrucciones sobre cómo utilizar determinadas funcionalidades de la app (creación de reportes y comentarios, uso del mapa, etc). Internamente, el asistente de IA utiliza un bucle agéntico para que el LLM de Mistral pueda refinar la respuesta final en un máximo de 10 iteraciones, pudiendo hacer uso de una serie de tools o funciones que realizan operaciones de lectura sobre la base de datos o peticiones a las APIs de AEMET / OpenMeteo para obtener la información solicitada por el usuario.

En la primera iteración del bucle agéntico, se le envía al LLM el contexto de la aplicación, las salvaguardas de seguridad (qué tipo de preguntas puede responder y cuáles no), y el prompt del usuario (el LLM determinará automáticamente la relevancia de dicho prompt, y lo rechazará si no está relacionado con ninguno de los temas permitidos). Tras cada iteración, el LLM decide si ya tiene suficiente información para devolver una respuesta consistente al usuario, o si necesita más datos (en cuyo caso solicitará utilizar las tools disponibles).

Otro aspecto a resaltar de la implementación es que el asistente de IA mantiene el contexto de la conversación con el usuario a lo largo de cada sesión. Esto quiere decir que, mientras el usuario mantenga la sesión iniciada en la app, puede cerrar y abrir nuevas pestañas y el chatbot será consciente de todo lo que se ha hablado desde la última vez que se inició sesión hasta el momento. Cuando la cierre, dicho contexto será reseteado (se eliminará).

En cuanto a la seguridad de la API del chatbot, se ha implementado un sistema de limitación de tasa de solicitudes a todos los usuarios no administradores. Dicho límite se ha establecido a 10 peticiones por día, y se resetea a las 12 de la noche mediante una tarea de node-cron.

10. Descripción de la validación y pruebas realizadas

La validación del back-end se implementa con Zod mediante esquemas específicos para cada endpoint, como inicio de sesión, registro o recuperación de contraseña. Estos esquemas se aplican en un middleware de validación antes de llegar al controlador. Si los datos enviados no cumplen las reglas definidas, Zod detecta el error y el middleware responde directamente con un código HTTP 400, ya que la petición es incorrecta por formato o contenido. Este tipo de respuesta pertenece a la familia de errores 4xx, mientras que otros códigos como 401 o 403 se reservan para problemas de autenticación o permisos, y los 5xx para errores internos del servidor. En las pruebas se comprobó que las peticiones válidas devuelven respuestas correctas y que las inválidas generan el 400 esperado con el mensaje de validación correspondiente.

Cumpliendo con los requisitos obligatorios del proyecto, se ha implementado el testing End-to-End (E2E) en el front-end utilizando Cypress. Para ello, se han desarrollado dos pruebas principales. La primera prueba evalúa la autenticación validando el flujo completo de login. Se comprueba el correcto renderizado del formulario, el inicio de sesión exitoso con la correspondiente redirección al mapa y el guardado del token de sesión, además del manejo de errores en la interfaz cuando se introducen credenciales incorrectas.

La segunda prueba E2E se centra en la creación de reportes, evaluando el flujo de interacción de un usuario autenticado. Durante este test, Cypress intercepta las llamadas de datos (como zonas, categorías y clima) para controlar la interfaz gráfica. Posteriormente, captura la petición de creación del reporte para verificar que las cabeceras incluyan el token de autorización adecuado y que se estén enviando los datos correctos de la zona, la categoría y la descripción. Finalmente, la prueba simula una respuesta de éxito del servidor para asegurar que el front-end cierra el modal correctamente y muestra la confirmación al usuario.

11. Mejoras implementadas

Se han incluido las siguientes mejoras opcionales:

- Permitir el **login con dos sistemas externos**, además del local (Google y Github).
 - **Descripción:** Se integró el inicio de sesión con Google y GitHub junto al sistema local (email/contraseña), simplificando el registro y delegando la seguridad de las credenciales a proveedores robustos.
 - **Implementación:**
 - **API y seguridad:** Se crearon los endpoints /login-google y /login-github en Swagger. Ambos validan los tokens y códigos de autorización en el

backend de forma segura mediante google-auth-library y peticiones asíncronas (fetch) a las APIs oficiales.

- **Lógica híbrida:** Los servicios dedicados verifican si el correo o ID externo ya existen en MongoDB. Si existen, vinculan la cuenta local con el proveedor; si no, registran automáticamente un nuevo perfil con los datos públicos del usuario.
 - **Sesión:** Tras la validación, el backend genera un JWT unificado que el frontend almacena en localStorage para proteger las rutas de la aplicación de manera idéntica al flujo tradicional.
- **Utilización de la herramienta de integración continua (CI) GitHub Actions en el desarrollo** para automatizar las tareas de compilación, pruebas y despliegue de código. Al tener dos repositorios diferentes se optó por separar la integración continua de ambos en flujos de trabajo independientes el uno del otro:

- **Repositorio backend:**

La integración continua en el backend es desencadenada automáticamente cada vez que se realiza un push a la rama principal. El proceso de compilación valida el código contra múltiples versiones de Node.js, descargando el código del repositorio, instalando el gestor de paquetes pnpm y todas las dependencias del proyecto.

La fase de despliegue depende de que la base de compilación funcione correctamente, en caso de que no sea así el despliegue se omite. Para llevar a cabo el despliegue fue necesario conectar el repositorio de github con vercel y, posteriormente, configurar dentro de los secretos del repositorio variables como ORG_ID, PROJECT_ID que hacen referencia al proyecto de vercel y son necesarias para la correcta vinculación del repositorio con el proyecto.

- **Repositorio front-end**

Para la parte del frontend, el proceso de integración y despliegue continuo presentó un nivel de complejidad mayor debido a la necesidad de implementar pruebas E2E destinadas a verificar el correcto funcionamiento de ciertas funcionalidades de la aplicación web antes de autorizar su paso a producción.

Al igual que en el backend, este flujo automatizado se desencadena inmediatamente tras detectar un nuevo push en la rama principal .

El proceso de despliegue en la plataforma Vercel sigue una lógica similar a la empleada en el backend, requiriendo la vinculación del repositorio mediante las credenciales del proyecto (ORG_ID y PROJECT_ID). Aun así, con el objetivo de explorar y aprender maneras alternativas se modificó la gestión de las variables de entorno para la aplicación. En lugar de almacenar estas variables directamente en la infraestructura en la nube de Vercel, se optó por gestionarlas como secretos internos dentro del propio repositorio de GitHub con la funcionalidad de secrets del propio github. Durante el flujo de trabajo, el

sistema descarga la configuración del proyecto, válida e inyecta dinámicamente dichos secretos en los archivos locales.

Por otra parte, la validación se lleva a cabo mediante la integración de la herramienta Cypress dentro del ciclo de vida de github actions. Tras asegurar la instalación correcta, el flujo de trabajo inicializa un servidor temporal de desarrollo. Una vez que este servicio web se encuentra completamente activo y disponible, la plataforma automatiza la ejecución de las pruebas E2E diseñadas para el proyecto.

En caso de que alguna de estas pruebas automatizadas falle, el flujo de trabajo se interrumpe de inmediato, cancelando los pasos posteriores y garantizando así que nunca se despliegue en producción una versión del frontend que contenga errores funcionales.

12. Valoración global del proyecto

La valoración general del desarrollo de Meteo Map es muy positiva. El proyecto ha logrado crear una solución completa, sólida y muy funcional. Combina información meteorológica oficial y en tiempo real con la participación activa de la comunidad y la inteligencia artificial, cumpliendo con éxito sus objetivos.

La integración de servicios externos como AEMET y Open-Meteo fue un desafío por la limitación de la tasa de peticiones. Esto se resolvió de manera profesional mediante la implementación de sistemas de caché y sincronización en segundo plano. Así se garantiza la disponibilidad de los datos y se reduce la latencia general del sistema.

La incorporación del modelo de inteligencia artificial **Mistral** a través de un bucle agéntico aporta un gran valor a la aplicación. Esto permite que el asistente mantenga el contexto y ejecute herramientas directamente contra la base de datos o las APIs externas.

Todo este trabajo de desarrollo interno se complementa con una fuerte capa de seguridad. Incluye un sistema de autenticación híbrido con varios proveedores, como credenciales nativas y OAuth2 con Google y Github. A nivel visual, el despliegue de un frontend interactivo con mapas dinámicos y representación gráfica de datos ofrece a los usuarios una herramienta analítica completa, fluida y fácil de usar.

Durante el desarrollo, se han identificado ciertas limitaciones en los entornos gratuitos y en la infraestructura serverless utilizada. Sin embargo, estos obstáculos han servido como una gran oportunidad de aprendizaje. Han permitido al equipo diagnosticar problemas y proponer una hoja de ruta clara para la escalabilidad futura. En conclusión, Meteo Map es un software bien documentado y estructurado. Representa la consolidación de los conocimientos adquiridos y muestra un claro potencial para su uso en un entorno real.

13. Mejoras propuestas

Limitaciones conocidas de la aplicación

- **Dependencia y cuotas de la API de Unsplash:** La aplicación depende críticamente de Unsplash para asociar imágenes dinámicas a los reportes meteorológicos según las condiciones o ubicaciones. Al utilizar el entorno de desarrollo (Demo Tier), existe una **limitación estricta de 50 peticiones por hora**. Si se supera este umbral, la interfaz sufre una degradación visual al tener que recurrir a imágenes de respaldo (placeholders).
- **Sincronización en tiempo real:** Actualmente, las validaciones de la comunidad ("Confirmar" o "Desmentir") y la actualización de nuevos comentarios se apoyan en peticiones HTTP tradicionales o refrescos de estado. No existe un canal bidireccional que actualice el mapa instantáneamente cuando otro usuario genera una alerta.
- **Acoplamiento a APIs de terceros:** La aplicación tiene una alta dependencia de la disponibilidad inmediata de servicios externos (Google OAuth, GitHub, OpenAI, Unsplash y APIs meteorológicas). La caída de cualquiera de ellos compromete funcionalidades clave debido a la falta de un sistema de contingencia offline.

Mejoras propuestas

Si se reiniciara el desarrollo del proyecto con la experiencia adquirida, se implementarían los siguientes cambios estratégicos y arquitectónicos:

A. Optimización multimedia y almacenamiento propio

- **Gestión de imágenes propias y almacenamiento en la nube:** En lugar de depender exclusivamente de un banco de imágenes estáticas como Unsplash, se permitiría a los administradores subir fotografías reales, personalizadas y específicas de cada zona geográfica, utilizando servicios de almacenamiento en la nube o soluciones autoalojadas (como AWS S3 o Nextcloud). Esto garantizaría el control total sobre el contenido visual de la interfaz y eliminaría por completo las limitaciones de cuotas de las APIs de terceros.

B. Evolución arquitectónica e interactividad

- **Comunicación bilateral con WebSockets:** Se sustituiría el flujo de datos pasivo por conexiones persistentes. Así, cuando un usuario reporte un riesgo extremo en el mapa, este aparecería de forma inmediata en las pantallas de todos los usuarios conectados en ese radio geográfico sin necesidad de recargar.
- **Arquitectura de microservicios:** Separar el backend en componentes independientes: un microservicio para la ingesta de datos meteorológicos y procesamiento de IA, y otro exclusivo para la gestión de usuarios y autenticación, evitando que una sobrecarga en las peticiones de mapas afecte al login.

C. Infraestructura de alta disponibilidad y escalabilidad en la nube (AWS)

- **Migración de Vercel a Amazon Web Services (AWS):** Actualmente, la aplicación se encuentra desplegada en Vercel, lo cual es óptimo para la fase de prototipado y el frontend. Sin embargo, para el backend de Express presenta restricciones severas, como el límite de tiempo de ejecución en funciones serverless (que afecta a tareas pesadas de IA) y la imposibilidad de mantener procesos persistentes en memoria (necesarios para los cron jobs de node-cron o futuras conexiones por WebSockets).
- **Despliegue con balanceo de carga (ELB) y auto scaling:** Si se rediseñara la infraestructura para producción, se optaría por migrar el backend a contenedores de Docker desplegados en **AWS ECS (Elastic Container Service)** o instancias **AWS EC2**, situadas detrás de un **Application Load Balancer (ALB)**.